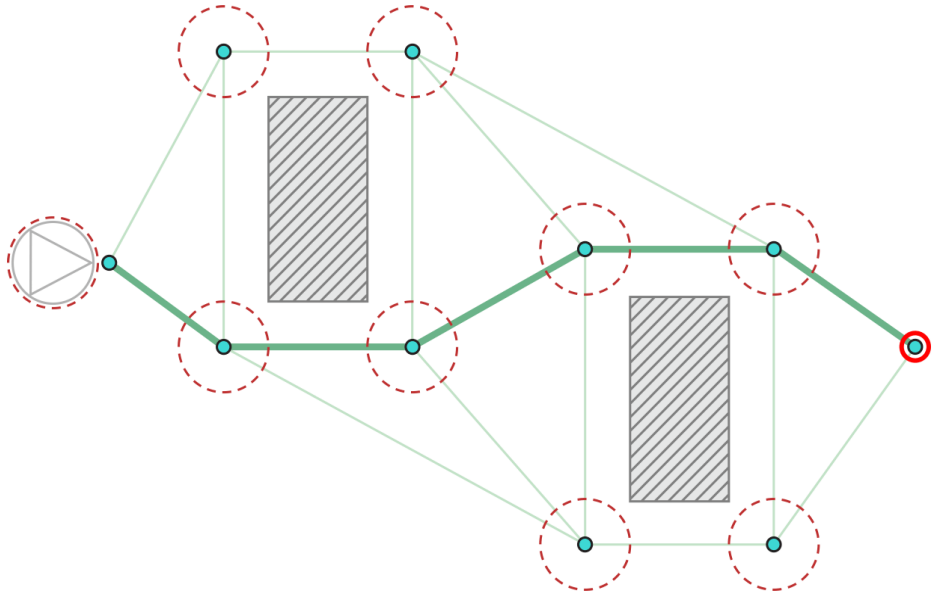

Robotyka – wyszukiwanie ścieżek

Zastosowania pathfinding'u

Algorytmy wyszukiwania ścieżek służą do wyznaczenia ścisłej trasy pomiędzy punktami uwzględniając warunki otoczenia (przeszkody, blokady). Wyznaczone trasy umożliwiają autonomicznym pojazdom/robotom ruch bez udziału czynnika ludzkiego.

Algorytmy te umożliwiają nam również między innymi wyszukiwanie najkrótszej trasy.



Przykładowe zastosowania algorytmów wyszukiwania ścieżek

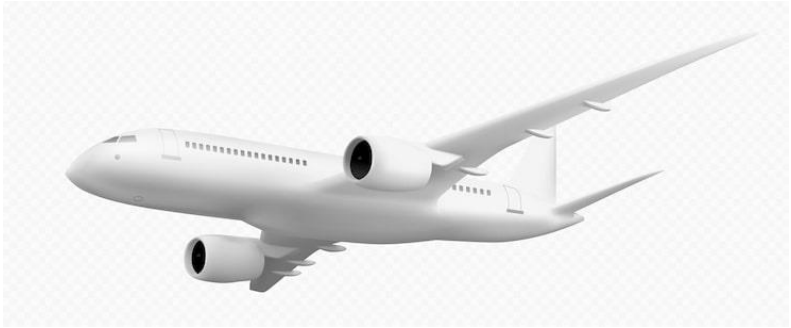
Przykładowe zastosowania algorytmów wyszukiwania ścieżek

Google Maps

Przykładowe zastosowania algorytmów wyszukiwania ścieżek

Google Maps

Przykładowe zastosowania algorytmów wyszukiwania ścieżek

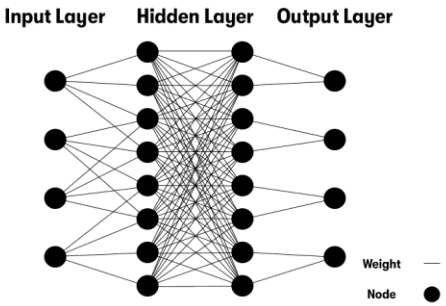


Przykładowe zastosowania algorytmów wyszukiwania ścieżek

Google Maps

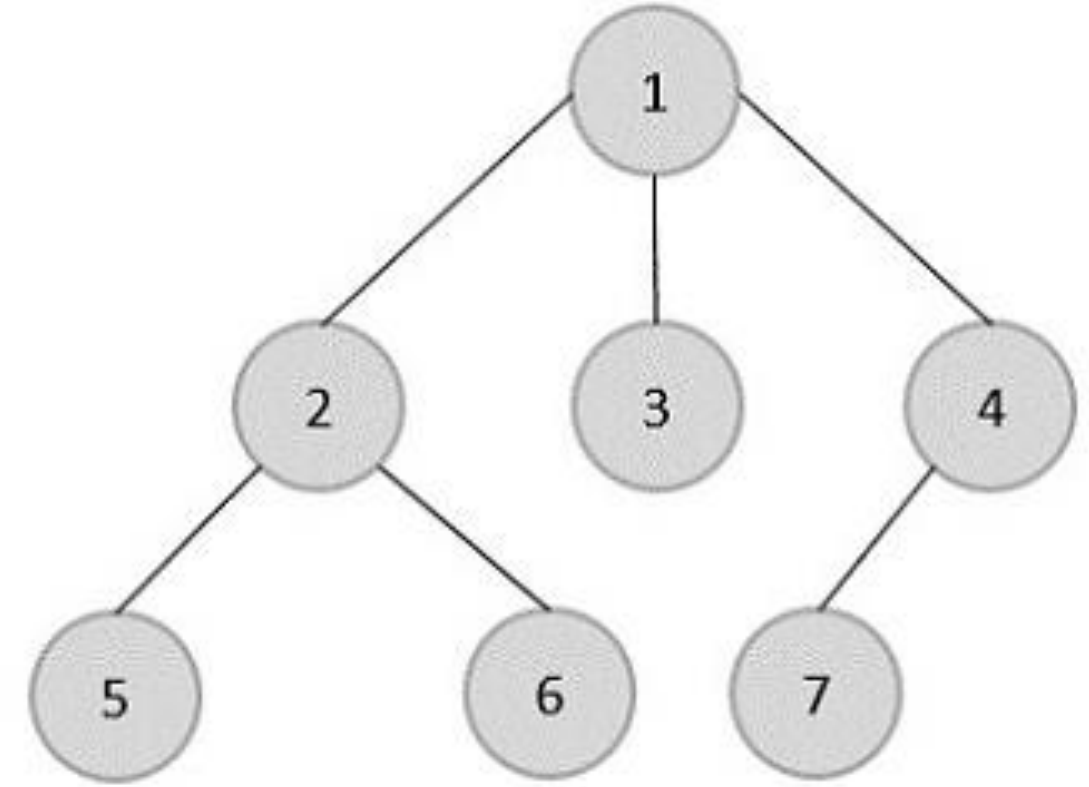
Przykładowe zastosowania algorytmów wyszukiwania ścieżek

Przykładowe zastosowania algorytmów wyszukiwania ścieżek



Struktury danych

Strukturą danych nazywamy sposób na przechowywanie danych w pamięci komputera. Określają one operacje, jakie można na nich wykonać.

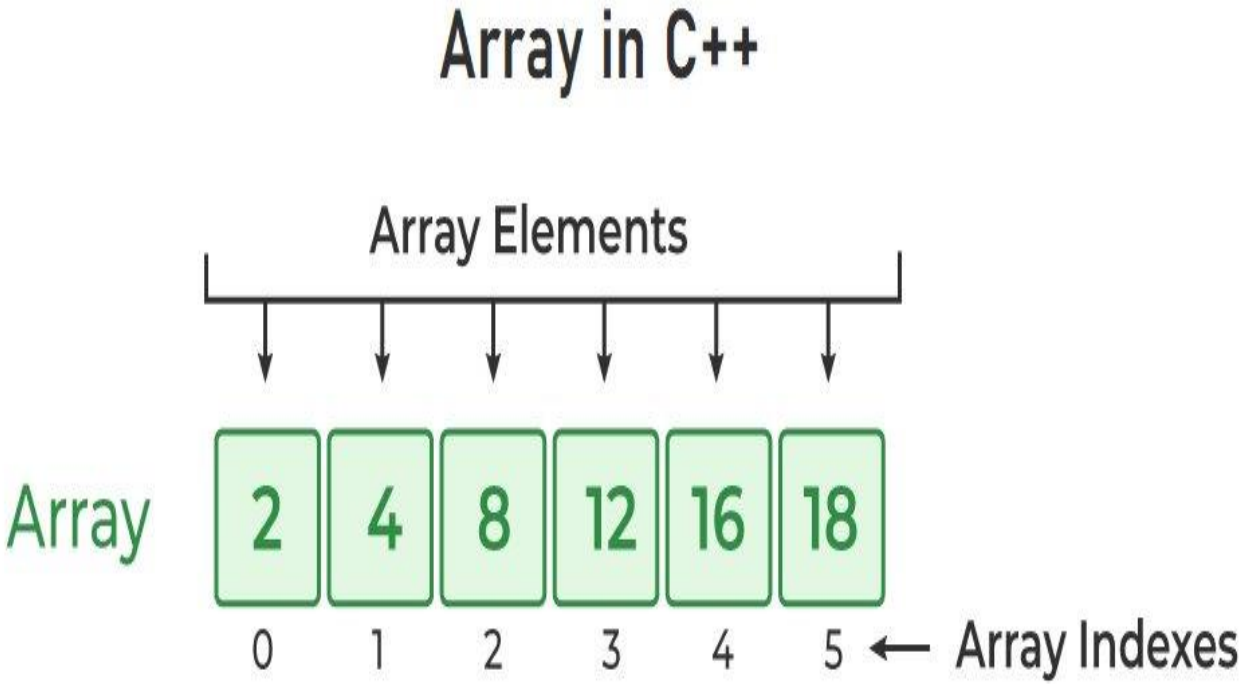


Tablica

-Dostęp do danych przez indeksy

-Stały rozmiar

-Ciągły obszar pamięci



Stos

-Zasada LIFO (Last In, First Out - ostatni wchodzi, pierwszy wychodzi)

-Dostęp tylko do elementu na szczycie stosu

-Dynamiczny rozmiar - może rosnąć i maleć w trakcie działania programu

Kolejka

-Zasada FIFO (First In, First Out – pierwszy wchodzi, pierwszy wychodzi)

-Dodawanie na końcu, usuwanie z początku

Zbiór

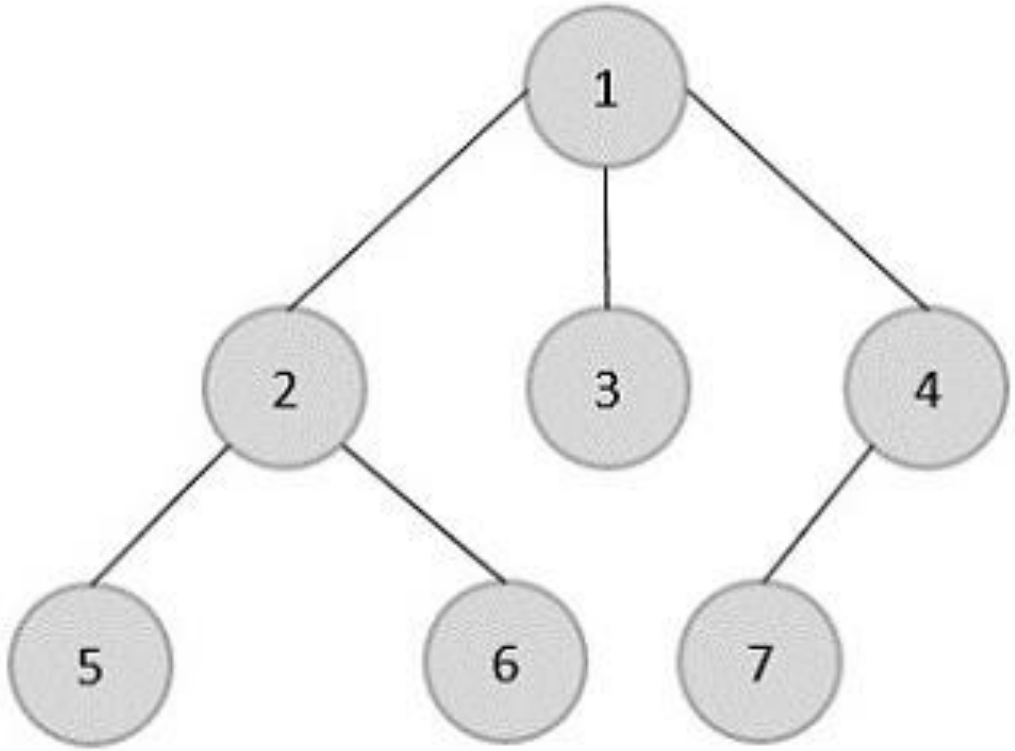
-Elementy są unikalne

-Brak określonego porządku

-Szybkie sprawdzenie istnienia elementu

Drzewo

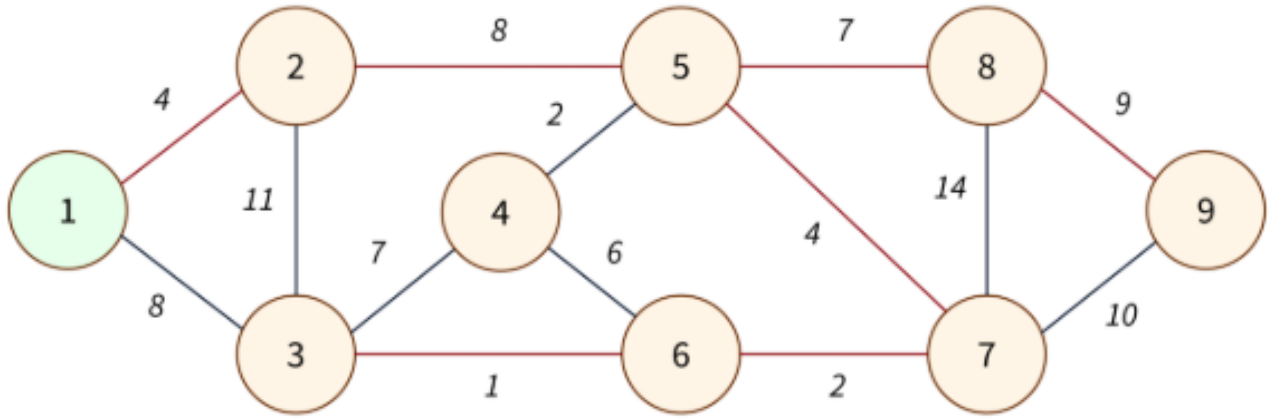
Struktura danych składająca się z wierzchołków i łączących je krawędzi. Jeden węzeł pełni rolę początku drzewa (root), a pozostałe tworzą hierarchię dzieci i rodziców. Drzewo jest szczególnym przypadkiem grafu.



Graf

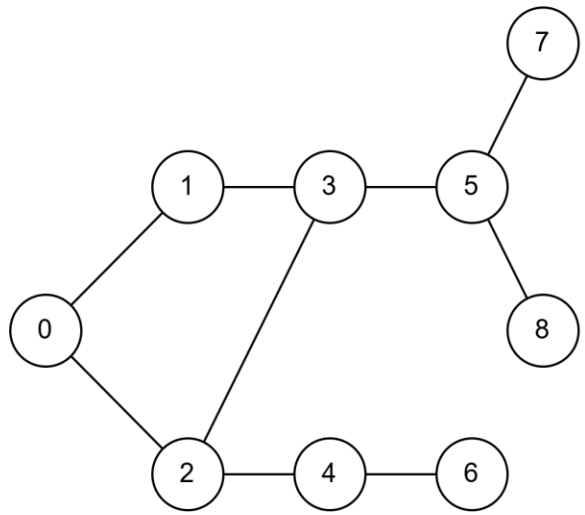
Struktura danych składająca się z wierzchołków i łączących je krawędzi. Grafy dzielimy na skierowane (określony kierunek krawędzi) i nieskierowane oraz ważone, jeśli poszczególne wierzchołki mają przypisane wagi.

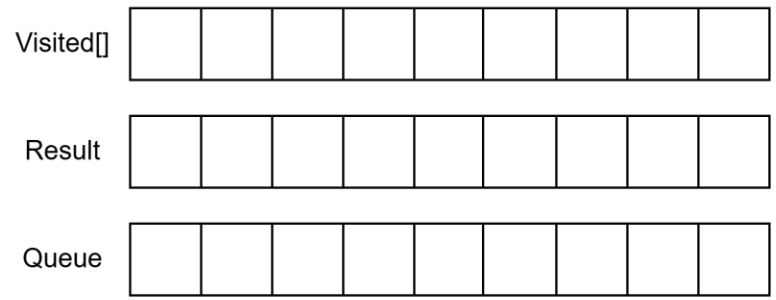
W algorytmice grafy służą do modelowania sieci, map, ścieżek i tym podobnych problemów.

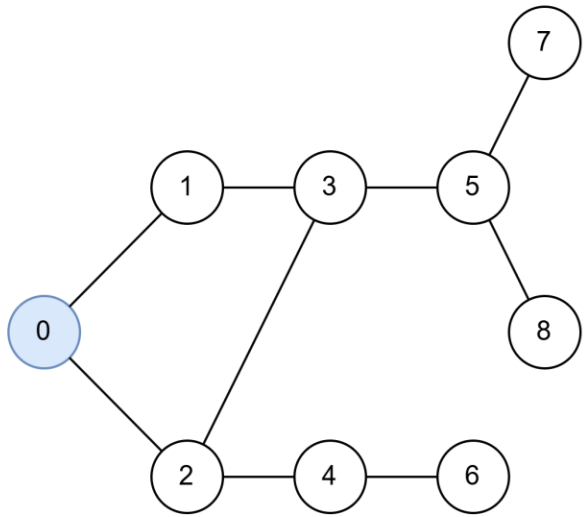


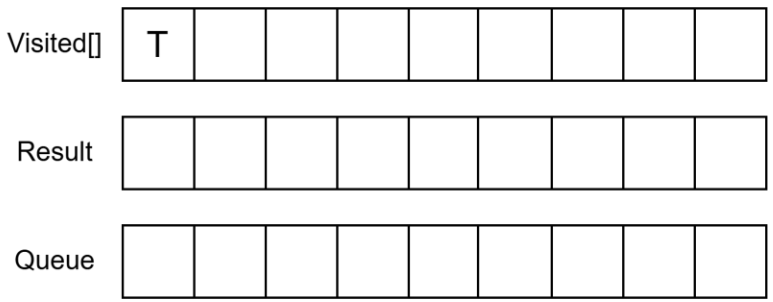
Algorytm przeszukiwania wszerz (BFS ang. Breadth-First Search)

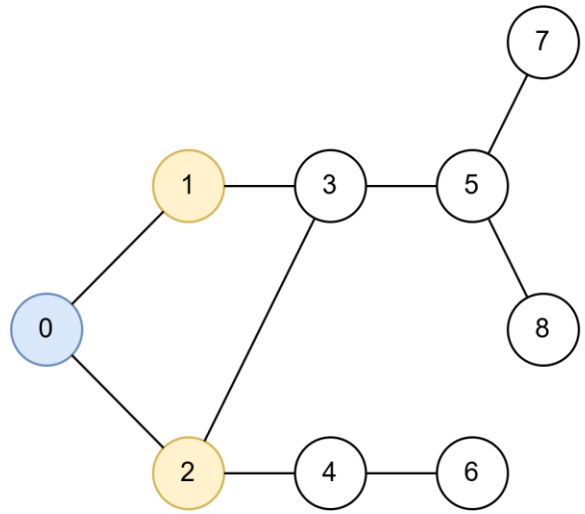
BFS to algorytm przeszukiwania grafu, który odwiedza wierzchołki warstwami, czyli najpierw wszystkie sąsiednie, potem ich sąsiadów itd. Zaczyna od wierzchołka startowego i korzysta z kolejki FIFO.


7
1
3
5
0
8
2
4
6

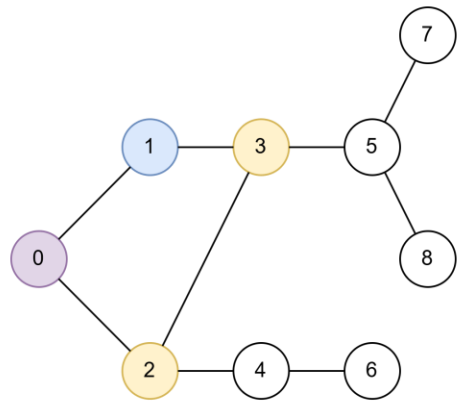

Visited[]
Result
Queue


7
1
3
5
0
8
2
4
6

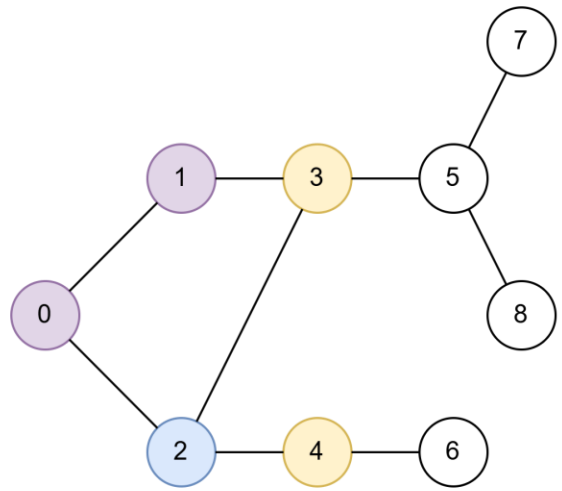

Visited[]
T
Result
Queue



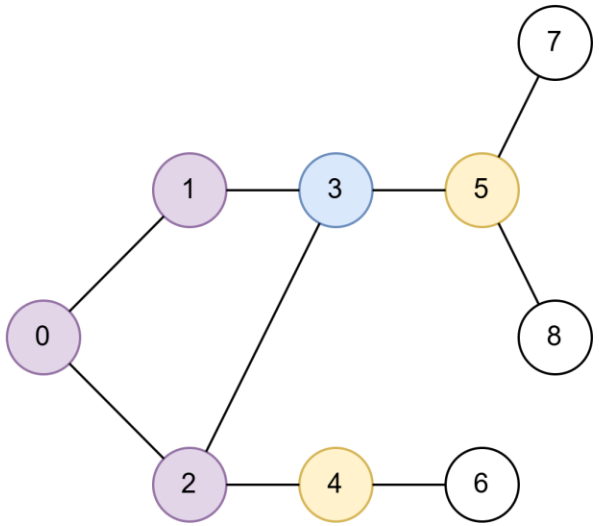
Visited[]	T	T	T						
Result									
Queue	1	2							



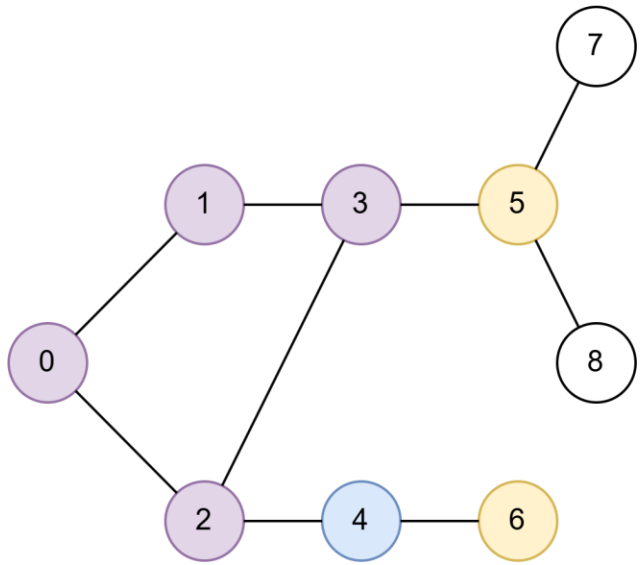
Visited[]	T	T	T	T					
Result	0								
Queue	2	3							



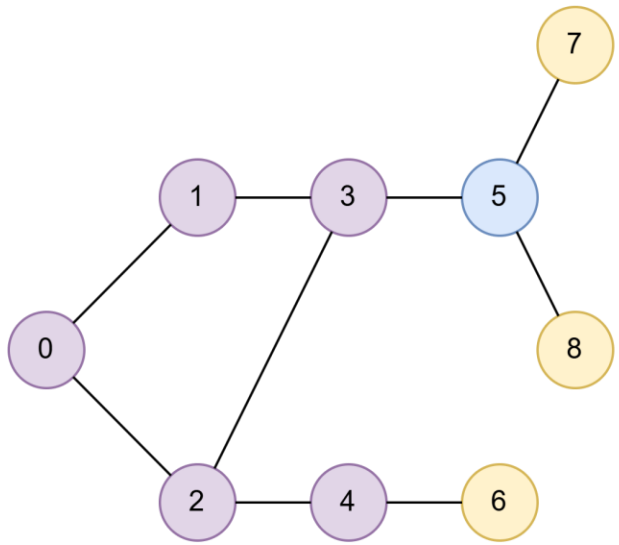
Visited[]	T	T	T	T	T				
Result	0	1							
Queue	3	4							



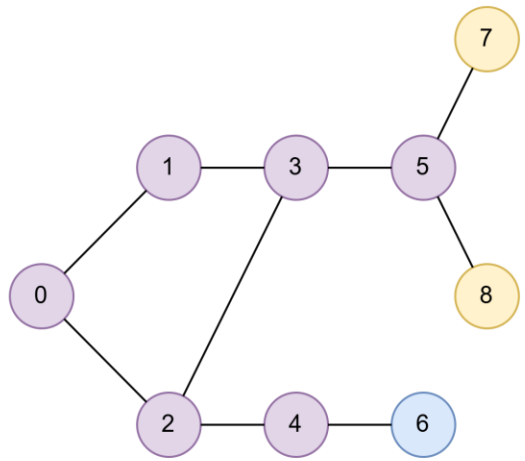
Visited[]	T	T	T	T	T	T			
Result	0	1	2						
Queue	4	5							



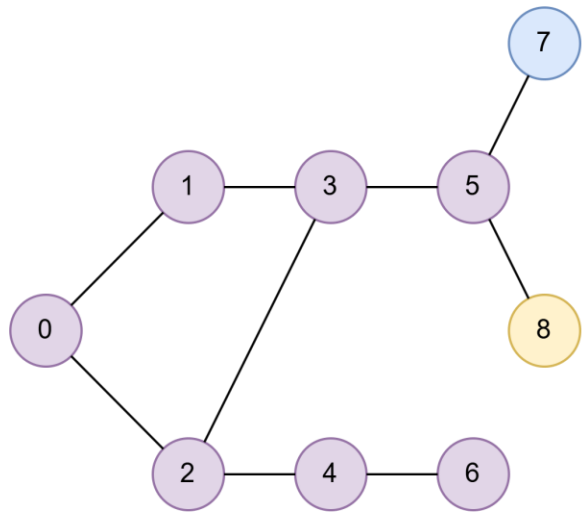
Visited[]	T	T	T	T	T	T	T		
Result	0	1	2	3					
Queue	5	6							



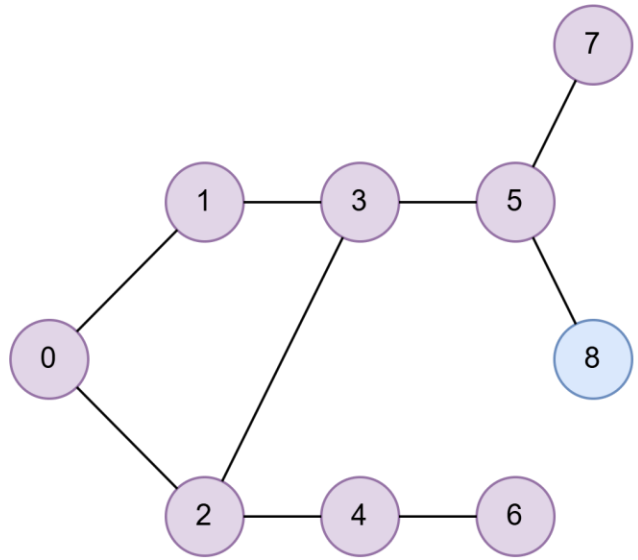
Visited[]	T	T	T	T	T	T	T	T	T
Result	0	1	2	3	4				
Queue	6	7	8						



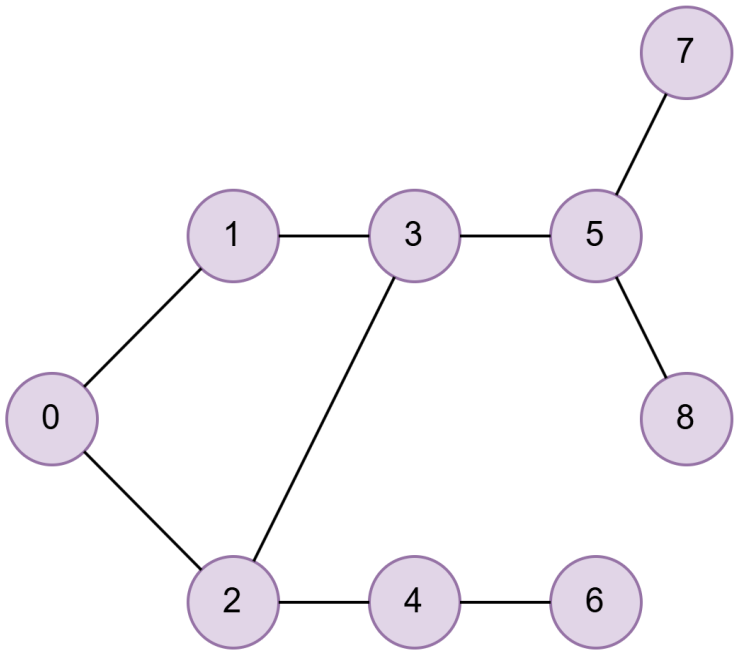
Visited[]	T	T	T	T	T	T	T	T	T
Result	0	1	2	3	4	5			
Queue	7	8							



Visited[]	T	T	T	T	T	T	T	T	T
Result	0	1	2	3	4	5	6		
Queue	8								



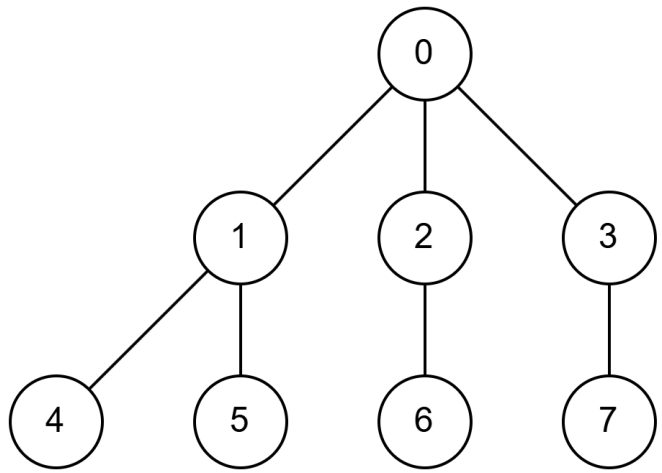
Visited[]	T	T	T	T	T	T	T	T	T
Result	0	1	2	3	4	5	6	7	
Queue									

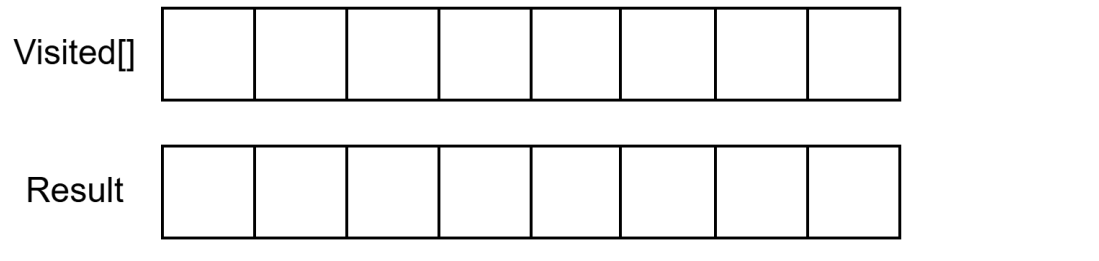


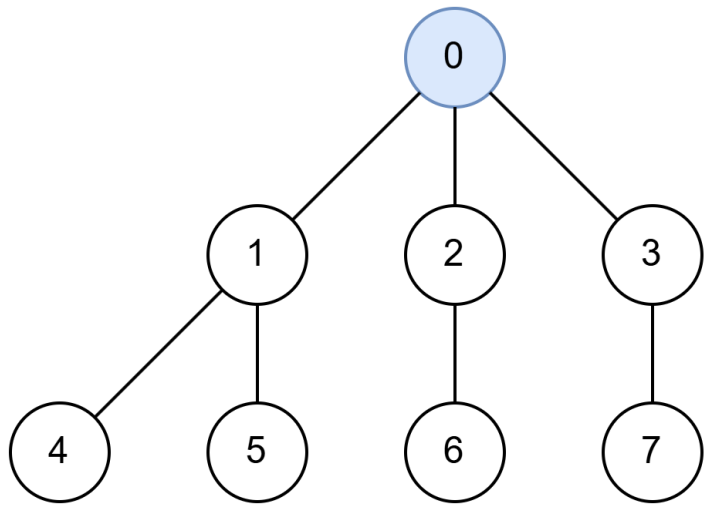
Visited[]	T	T	T	T	T	T	T	T	T
Result	0	1	2	3	4	5	6	7	8
Queue									

Algorytm przeszukiwania w głąb (DFS ang. Depth-First Search)

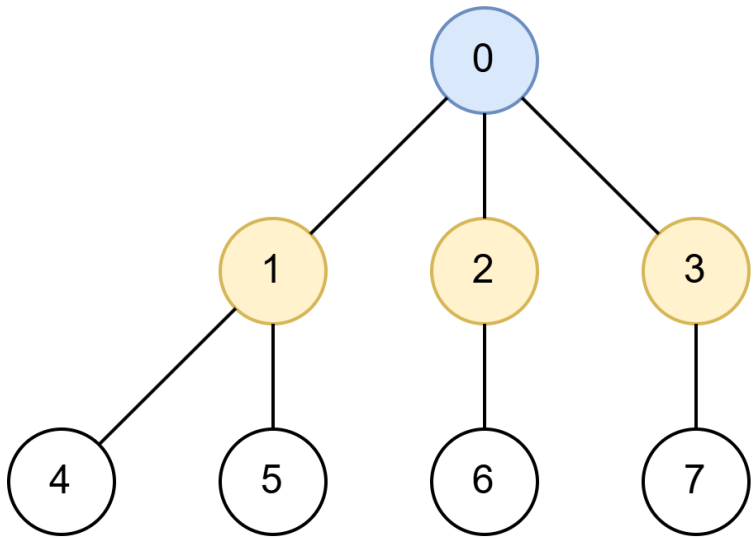
DFS to algorytm przeszukiwania grafu, który eksploruje jak najgłębiej jedną ścieżkę, zanim wróci i sprawdzi inne możliwości. Korzysta z stosu (kolejka LIFO), co powoduje, że kieruje się w głąb grafu aż do końca gałęzi.


0
1
2
3
4
5
6
7


Visited[]
Result
Stack



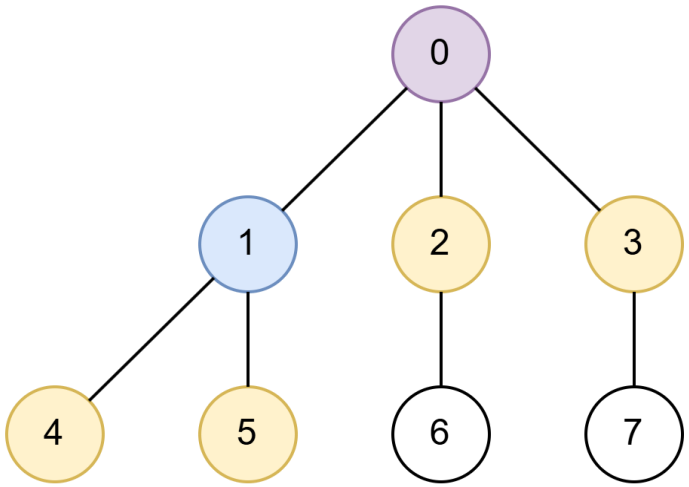
Stack



Visited[]	T	T	T	T				
Result								

1
2
3

Stack

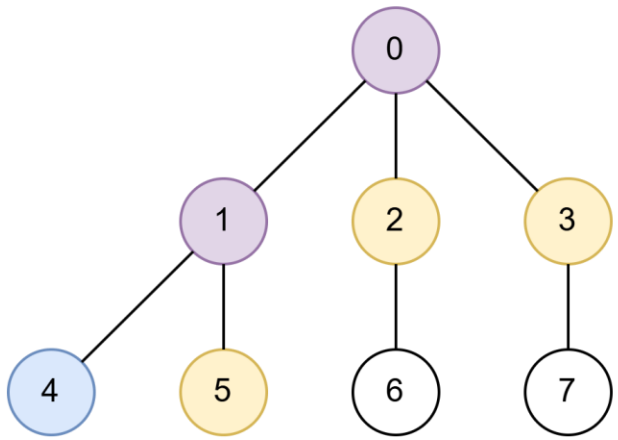


Visited[]								
	T	T	T	T	T	T		

Result								
	0							

4
5
~~1~~
2
3

Stack

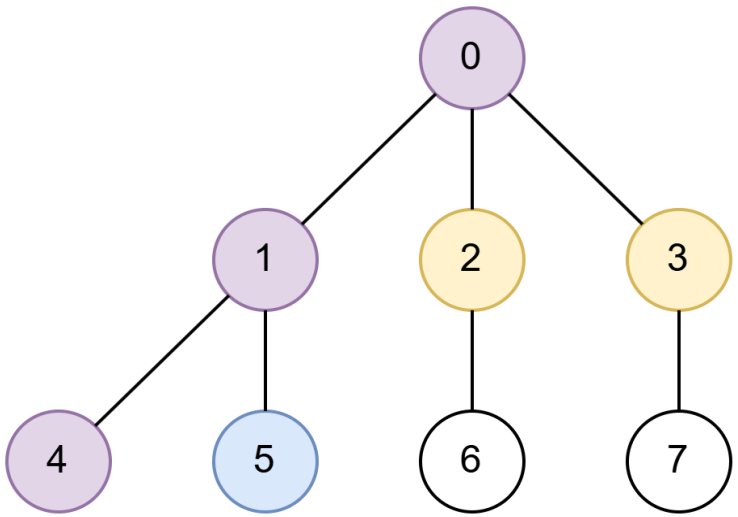


Visited[]	T	T	T	T	T	T		

Result	0	1						

~~4~~
5
2
3

Stack

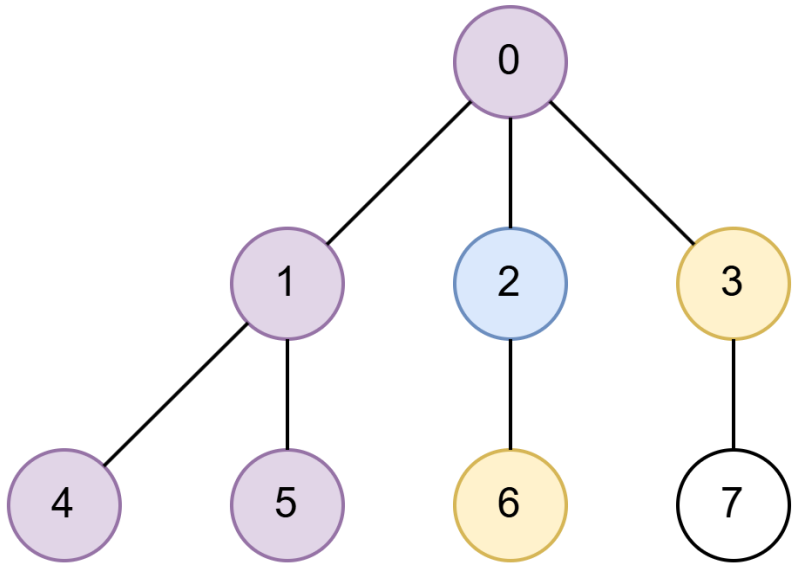


Visited[]								
	T	T	T	T	T	T		

Result								
	0	1	4					

~~5~~
2
3

Stack

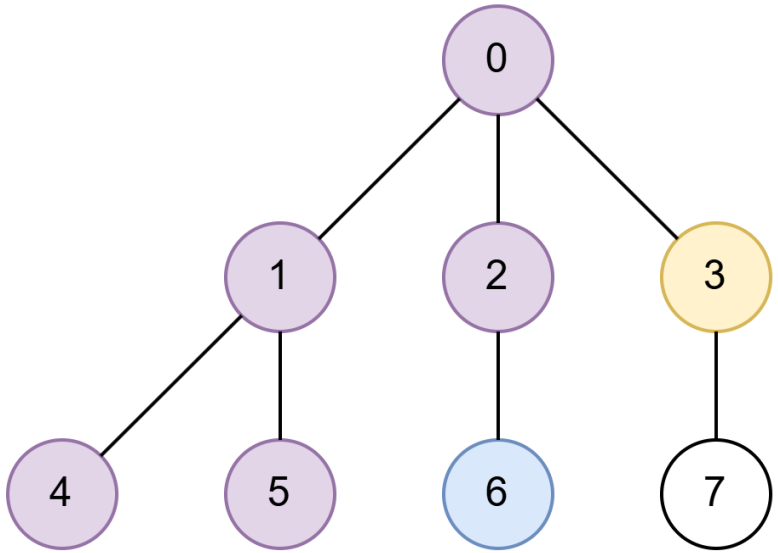


Visited[]	T	T	T	T	T	T	T	

Result	0	1	4	5				

6
~~2~~
3

Stack



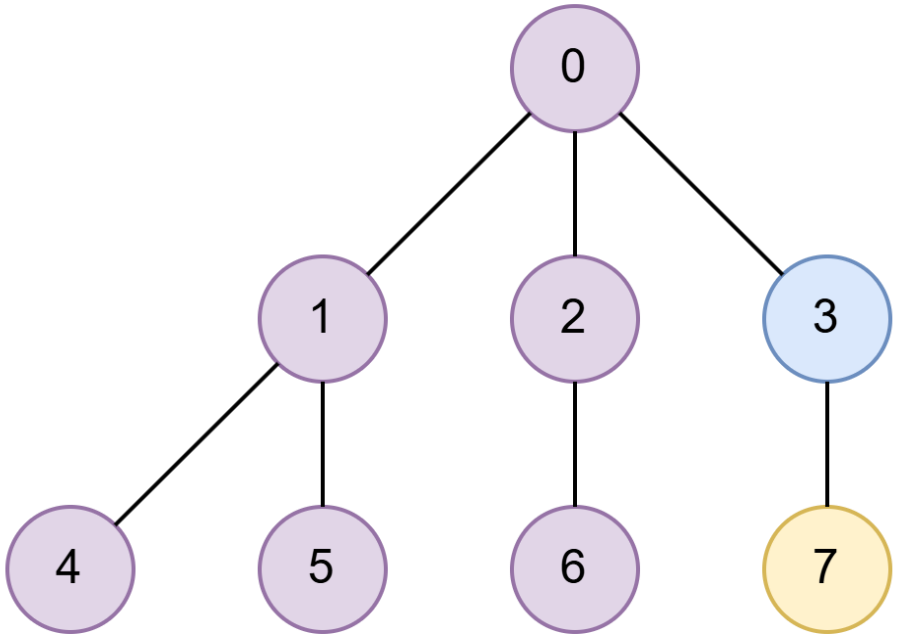
Visited[]	T	T	T	T	T	T	T	

Result	0	1	4	5	2			

~~6~~

3

Stack



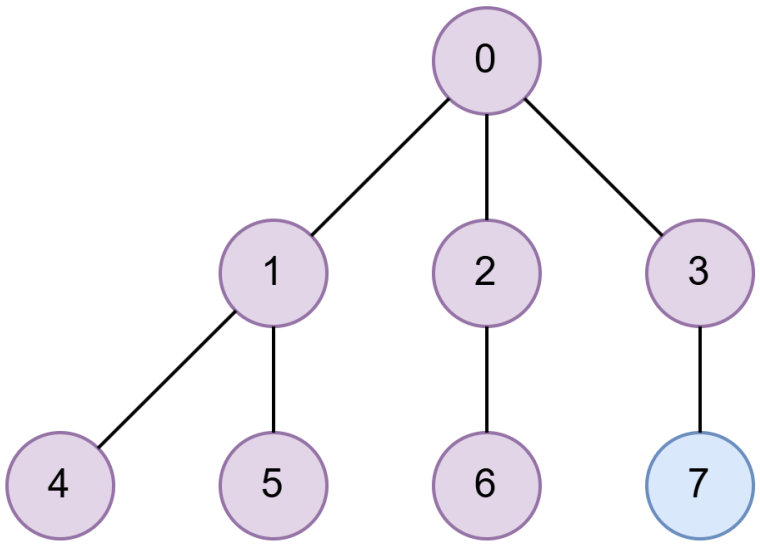
Visited[]	T	T	T	T	T	T	T	T

Result	0	1	4	5	2	6		

7

~~3~~

Stack

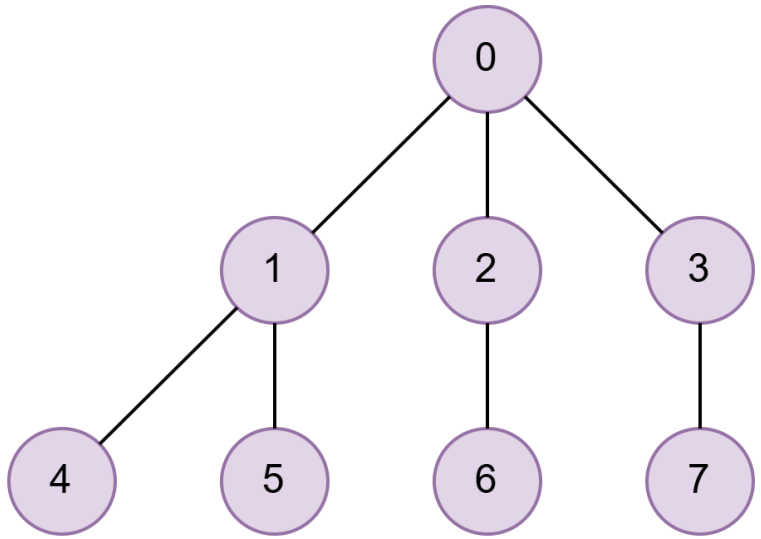


Visited[]	T	T	T	T	T	T	T	T

Result	0	1	4	5	2	6	3	

~~7~~

Stack



Visited[]

T	T	T	T	T	T	T	T

Result

0	1	4	5	2	6	3	7

Stack